Assuming that the `@customer` instance variable contains an instance of the `Customer` model, this will use `_customer.html.erb` to render it and will pass the local variable `customer` into the partial which will refer to the `@customer` instance variable in the parent view.

3.4.5 Rendering Collections

Partials are very useful in rendering collections. When you pass a collection to a partial via the `:collection` option, the partial will be inserted once for each member in the collection:

- index.html.erb
```
<h1>Products</h1>
<%= render partial: "product", collection: @products %>
```
- _product.html.erb
```
<p>Product Name: <%= product.name %></p>
```

When a partial is called with a pluralized collection, then the individual instances of the partial have access to the member of the collection being rendered via a variable named after the partial. In this case, the partial is `_product`, and within the `_product` partial, you can refer to `product` to get the instance that is being rendered.

There is also a shorthand for this. Assuming `@products` is a collection of `product` instances, you can simply write this in the `index.html.erb` to produce the same result:

```
<h1>Products</h1>
<%= render @products %>
```

Rails determines the name of the partial to use by looking at the model name in the collection. In fact, you can even create a heterogeneous collection and render it this way, and Rails will choose the proper partial for each member of the collection:

- index.html.erb
```
<h1>Contacts</h1>
<%= render [customer1, employee1, customer2, employee2] %>
```
- customers/_customer.html.erb
```
<p>Customer: <%= customer.name %></p>
```
- employees/_employee.html.erb
```
<p>Employee: <%= employee.name %></p>
```

In this case, Rails will use the customer or employee partials as appropriate for each member of the collection.

In the event that the collection is empty, `render` will return nil, so it should be fairly simple to provide alternative content.

```
<h1>Products</h1>
<%= render(@products) || "There are no products available." %>
```

3.4.6 Local Variables

To use a custom local variable name within the partial, specify the `:as` option in the call to the partial:

```
<%= render partial: "product", collection: @products, as: :item %>
```

With this change, you can access an instance of the `@products` collection as the `item` local variable within the partial.

You can also pass in arbitrary local variables to any partial you are rendering with the `locals: {}`option:

```
<%= render partial: "product", collection: @products,
           as: :item, locals: {title: "Products Page"} %>
```

In this case, the partial will have access to a local variable `title` with the value "Products Page".

Rails also makes a counter variable available within a partial called by the collection, named after the member of the collection followed by `_counter`. For example, if you're rendering `@products`, within the